Algoritmos I

Martín René Vilugrón
mrvilugron@unrn.edu.ar

UNRN
Universidad Nacional
de **Río Negro**

18 / 2026

r33

Estructuras de datos

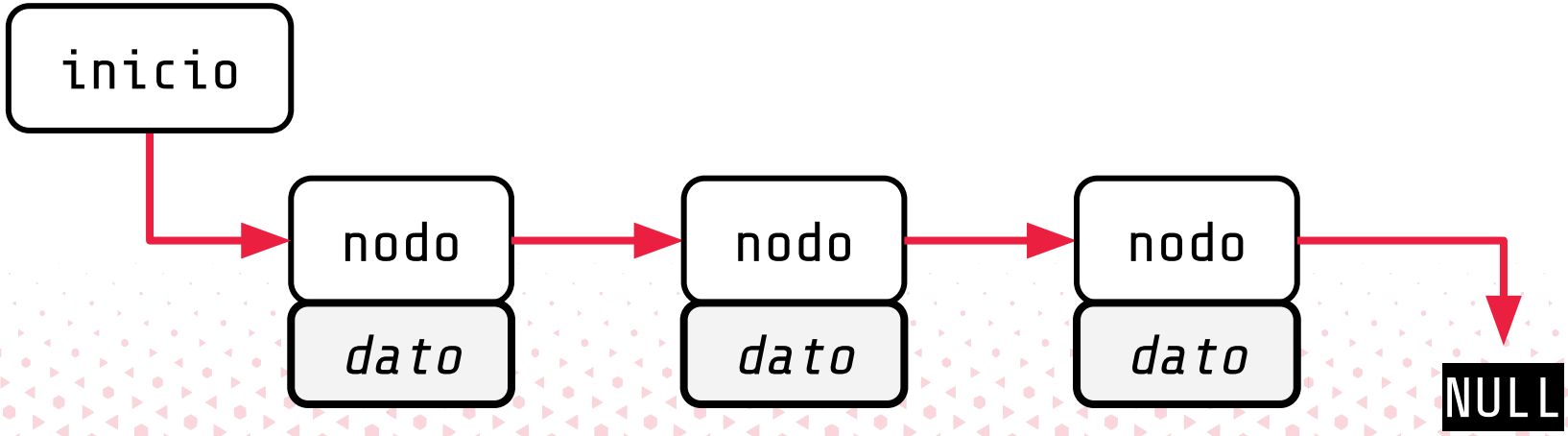
UNRN Universidad Nacional de Río Negro

Listas enlazadas

UNRN Universidad Nacional de Río Negro

{ Como los arreglos, pero *más flexibles* }

Lista enlazada *simple*



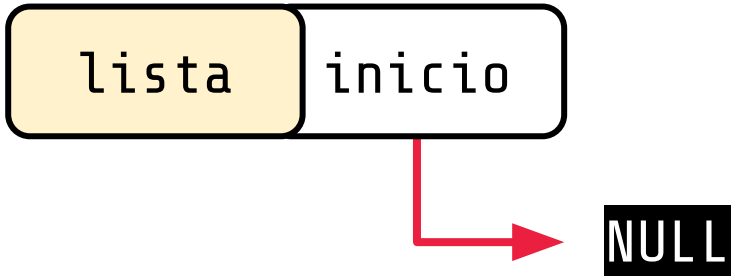
Que se implementa con esta estructura

```
typedef struct nodo
{
    struct nodo* siguiente;
    int valor;
}nodo_t;
```

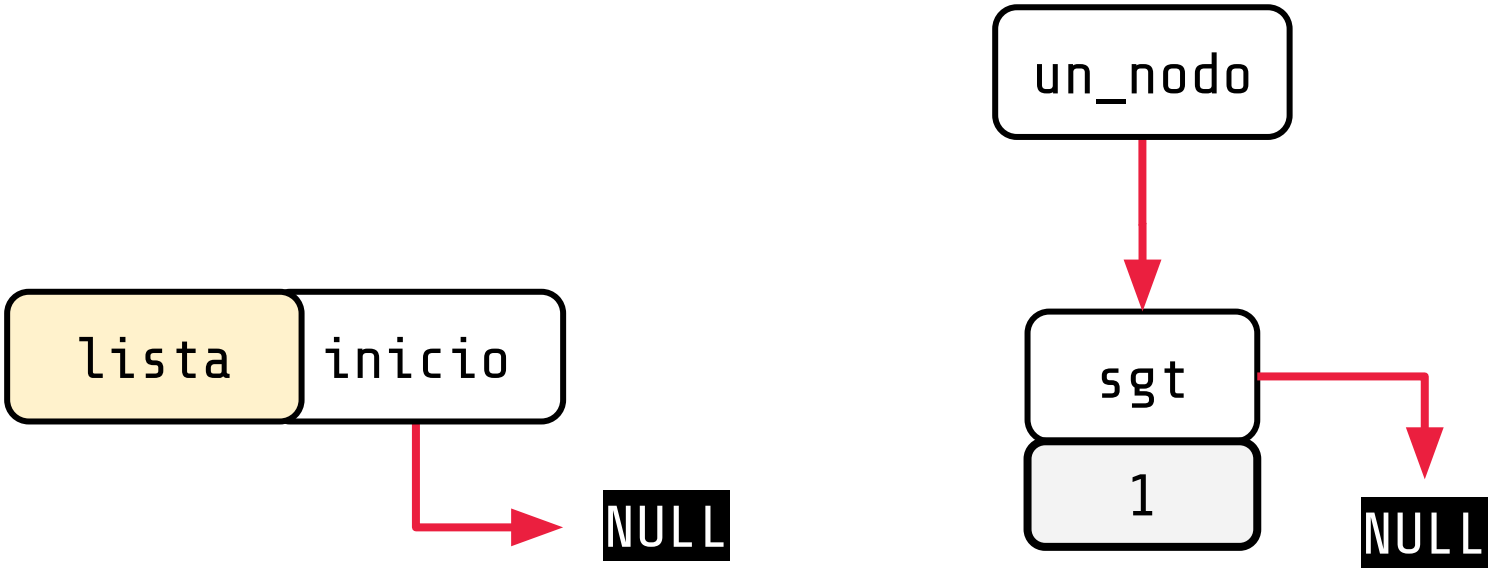
```
typedef struct
{
    struct nodo* inicio;
}lista_t;
```



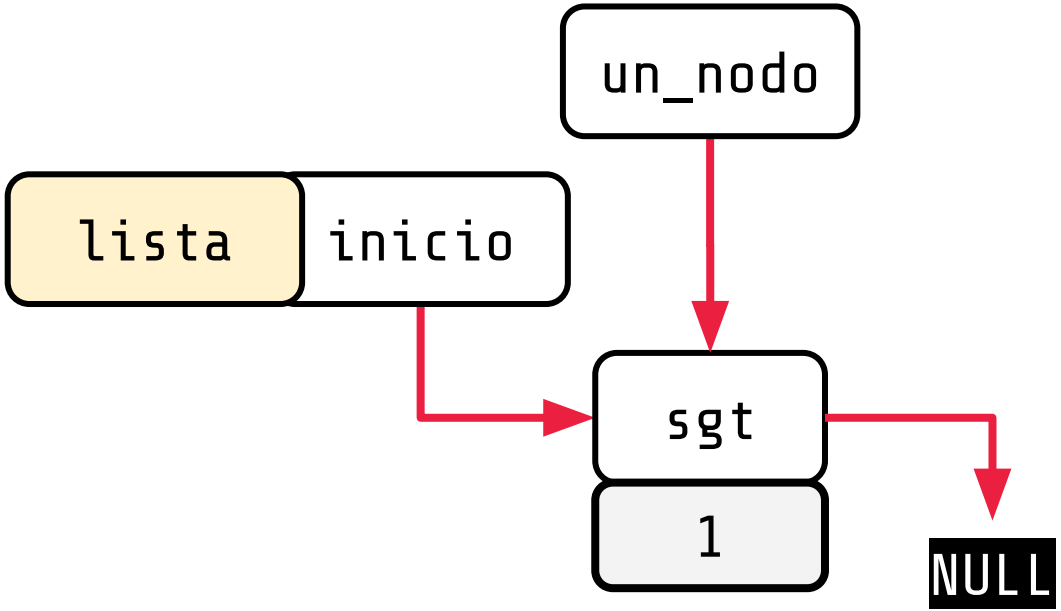
La construcción



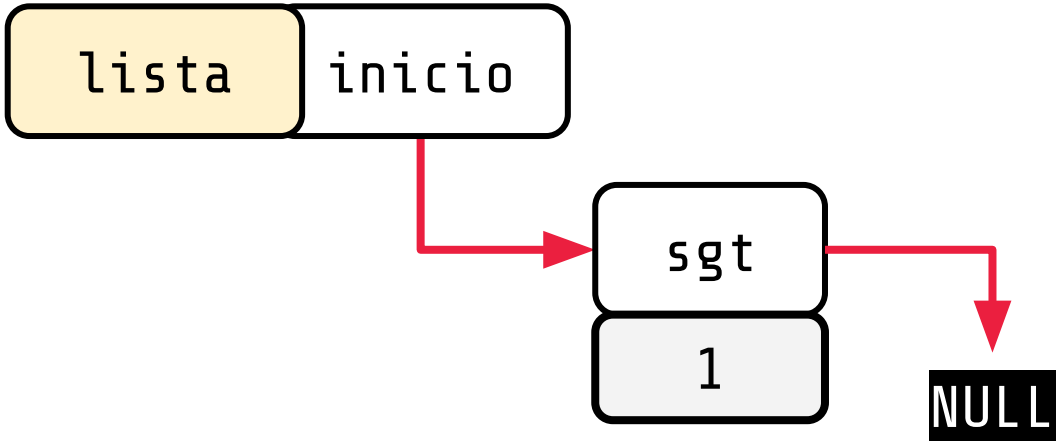
Inserción



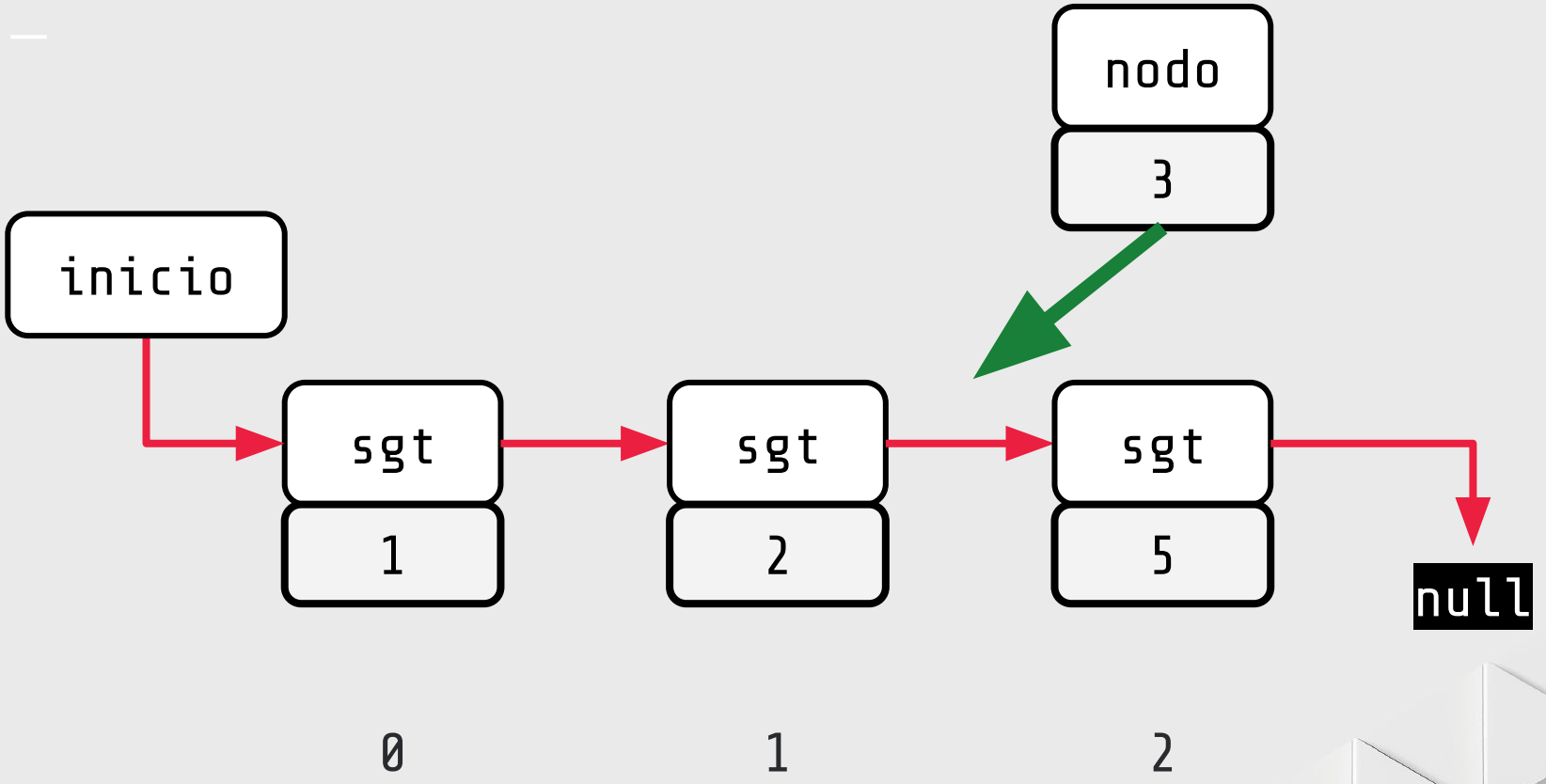
Y lo conectamos a la lista

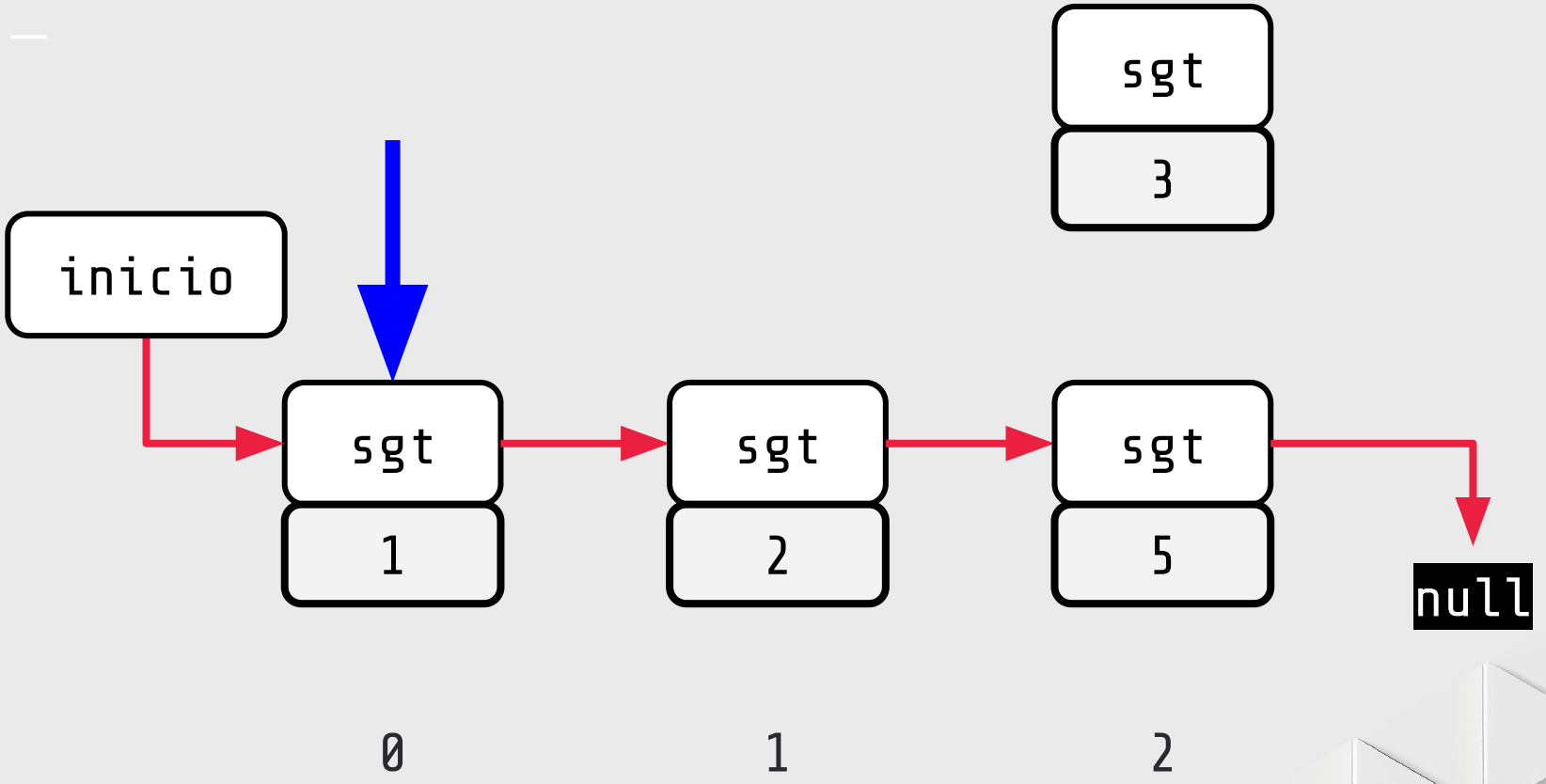


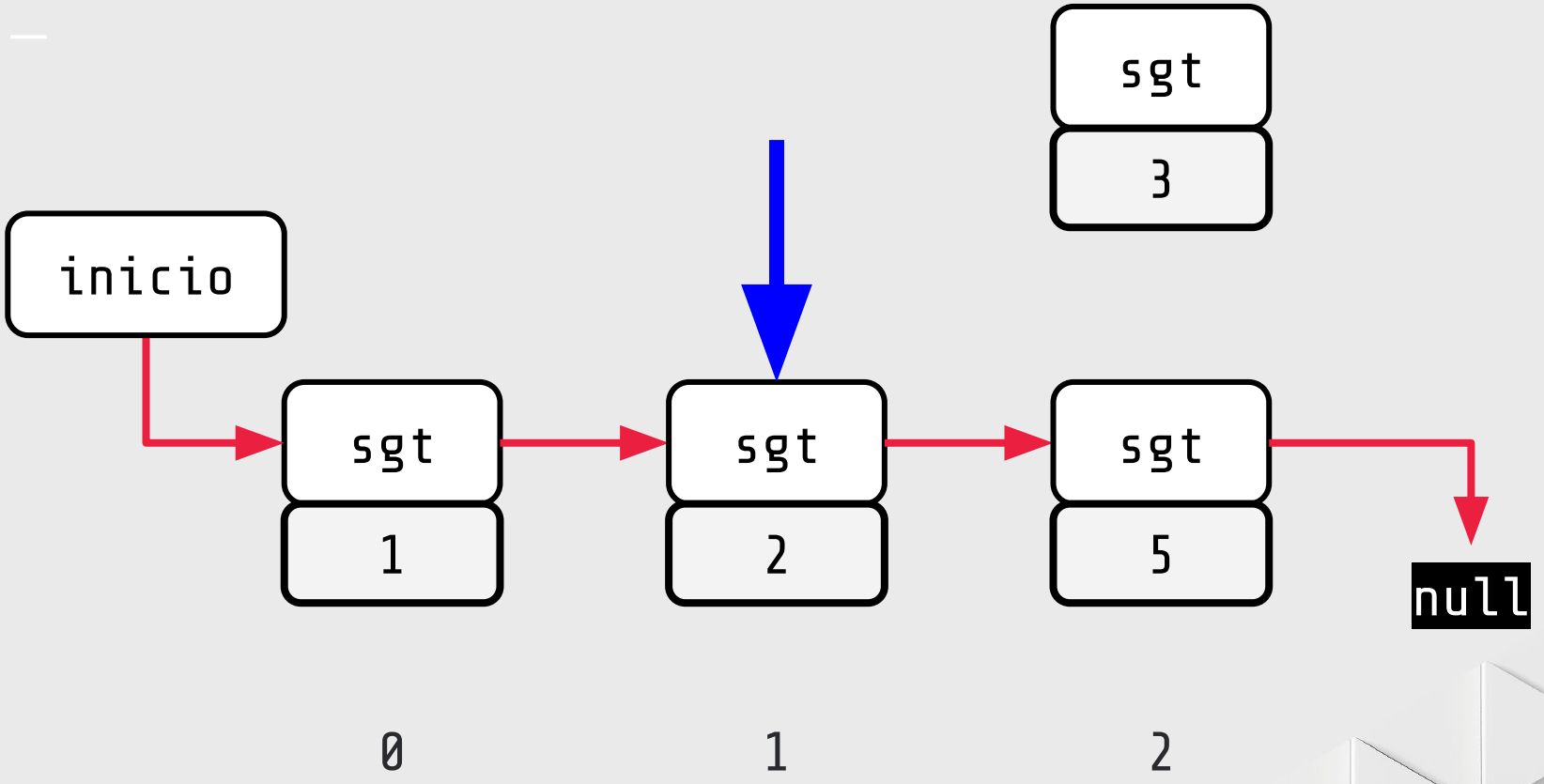
Y la lista queda con un nodo

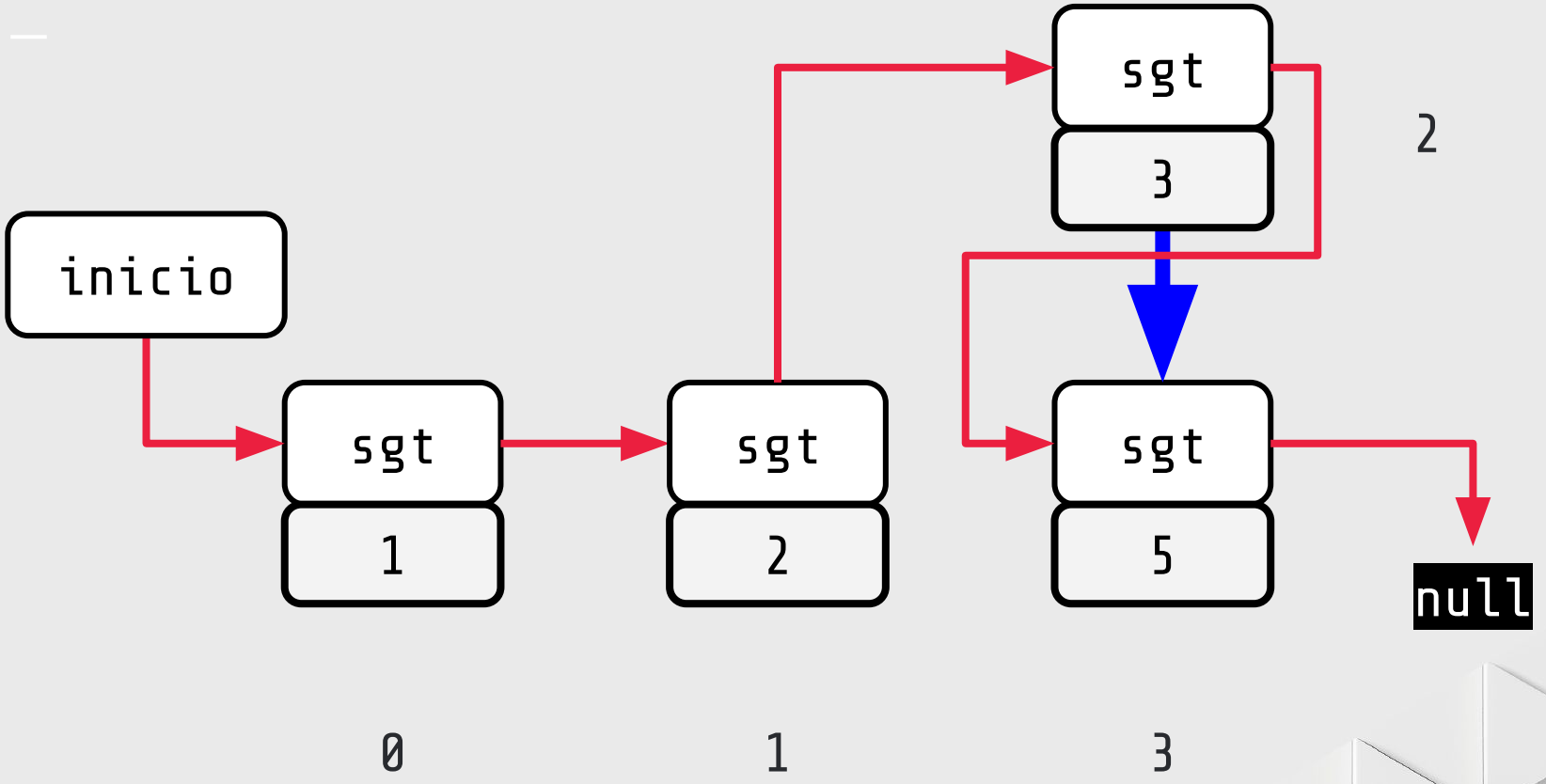


Una inserción en una lista más grande


nodo
3
inicio
sgt
1
sgt
2
sgt
5
null
0
1
2

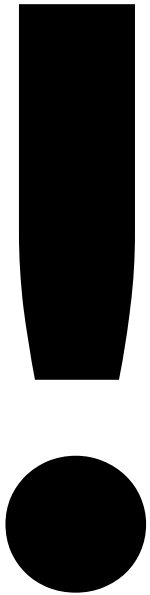

sgt
3
inicio
sgt
1
sgt
2
sgt
5
null
0
1
2


sgt
3
inicio
sgt
1
sgt
2
sgt
5
null
0
1
2

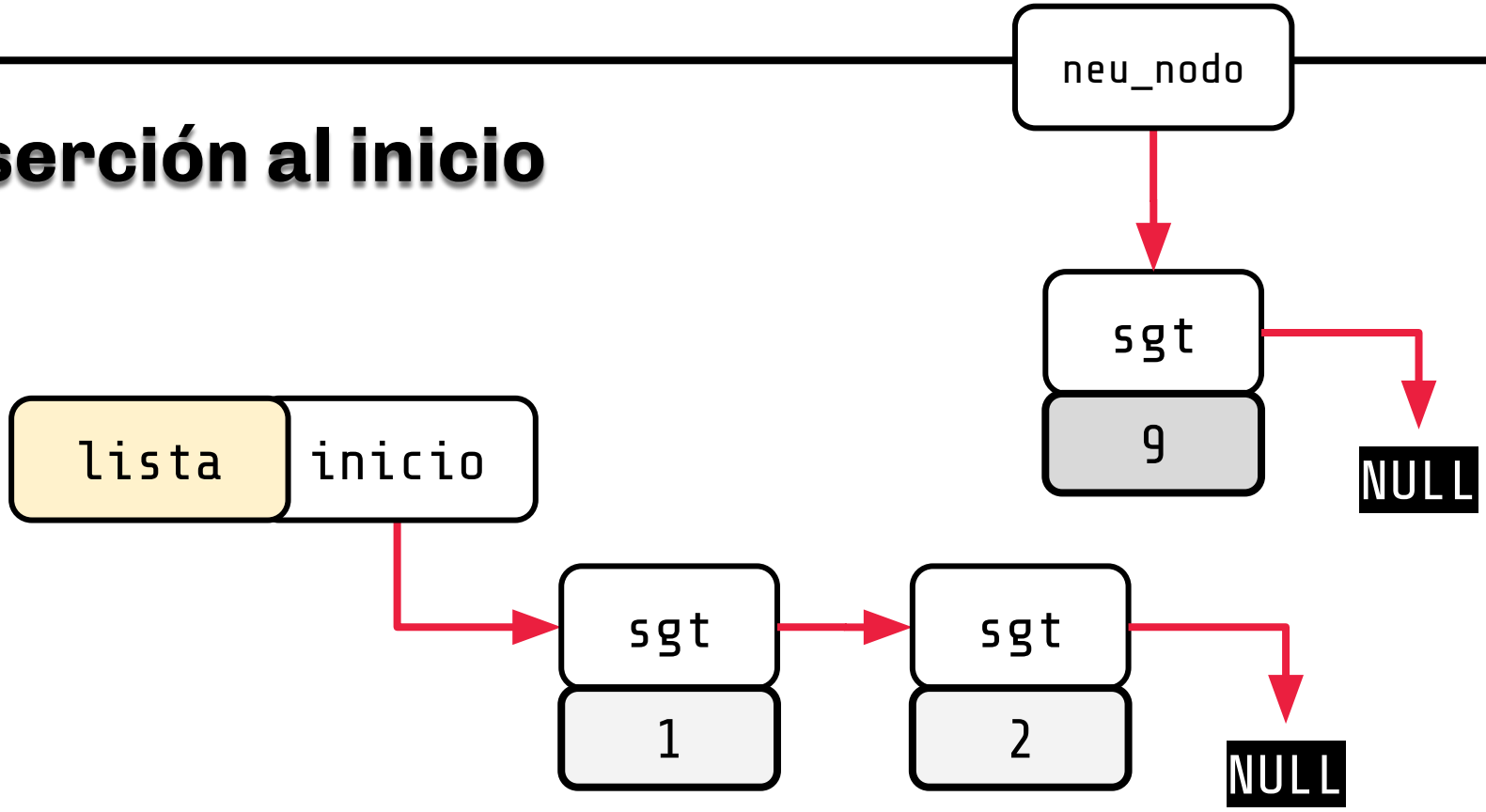

sgt
3
2
inicio
sgt
1
sgt
2
sgt
5
null
0
1
3



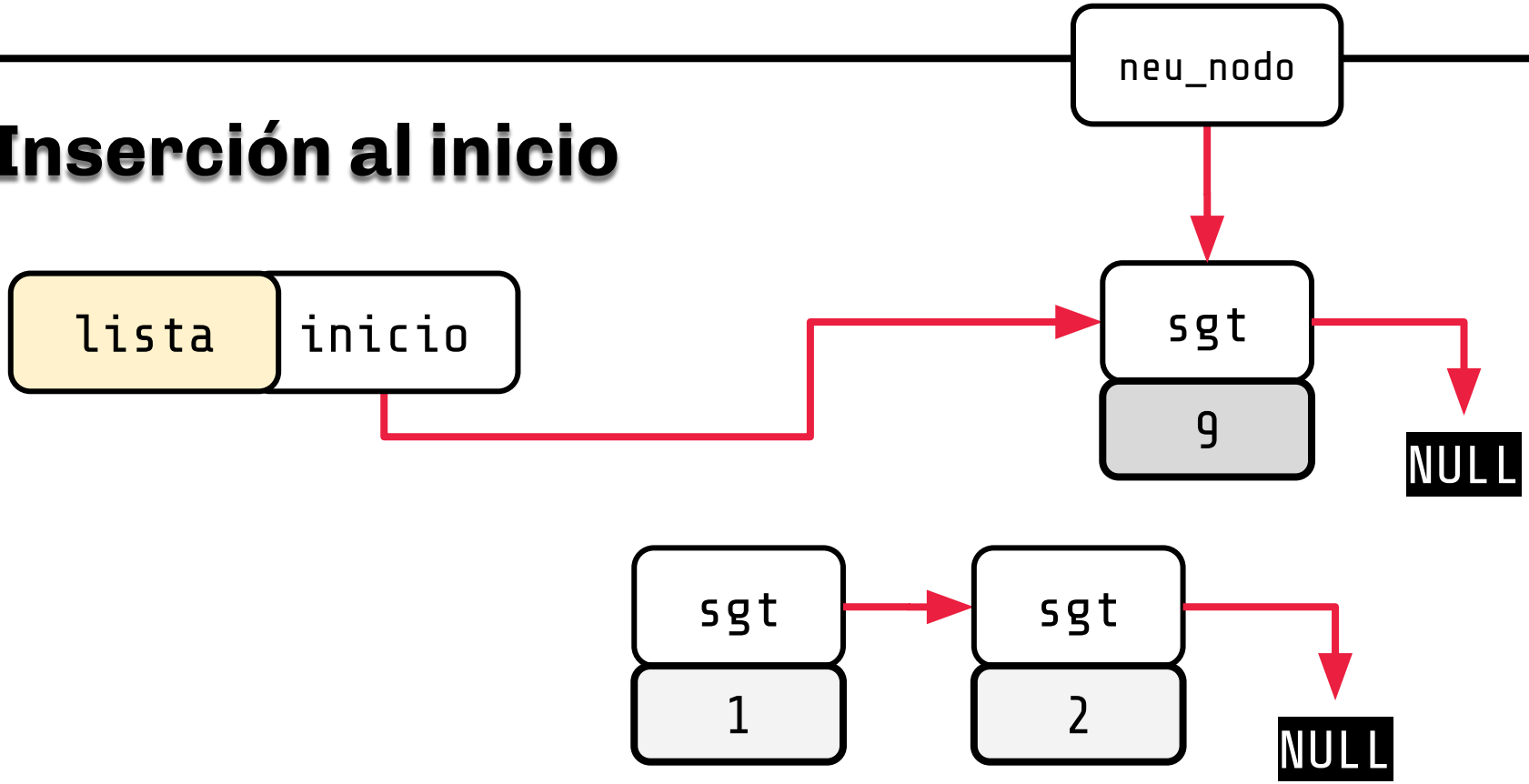
La gran ventaja



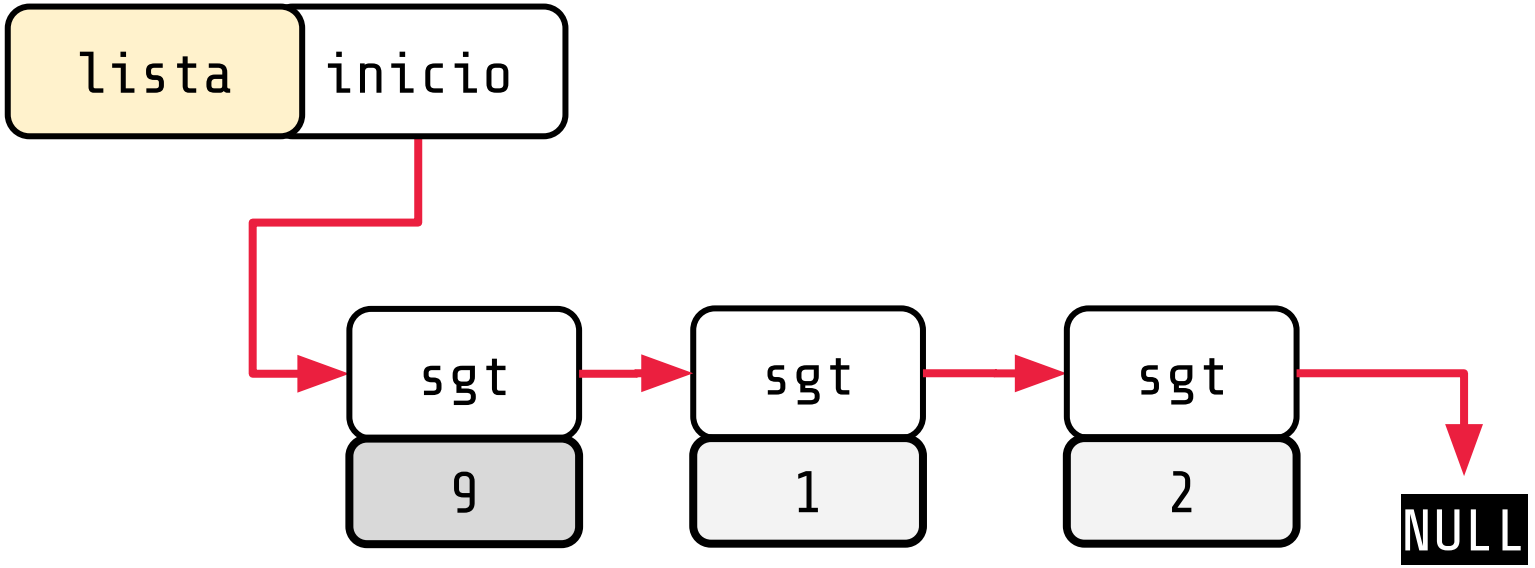
Inserción al inicio



Inserción al inicio

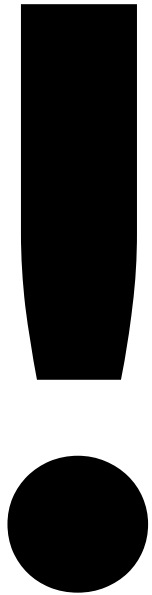


Inserción al inicio

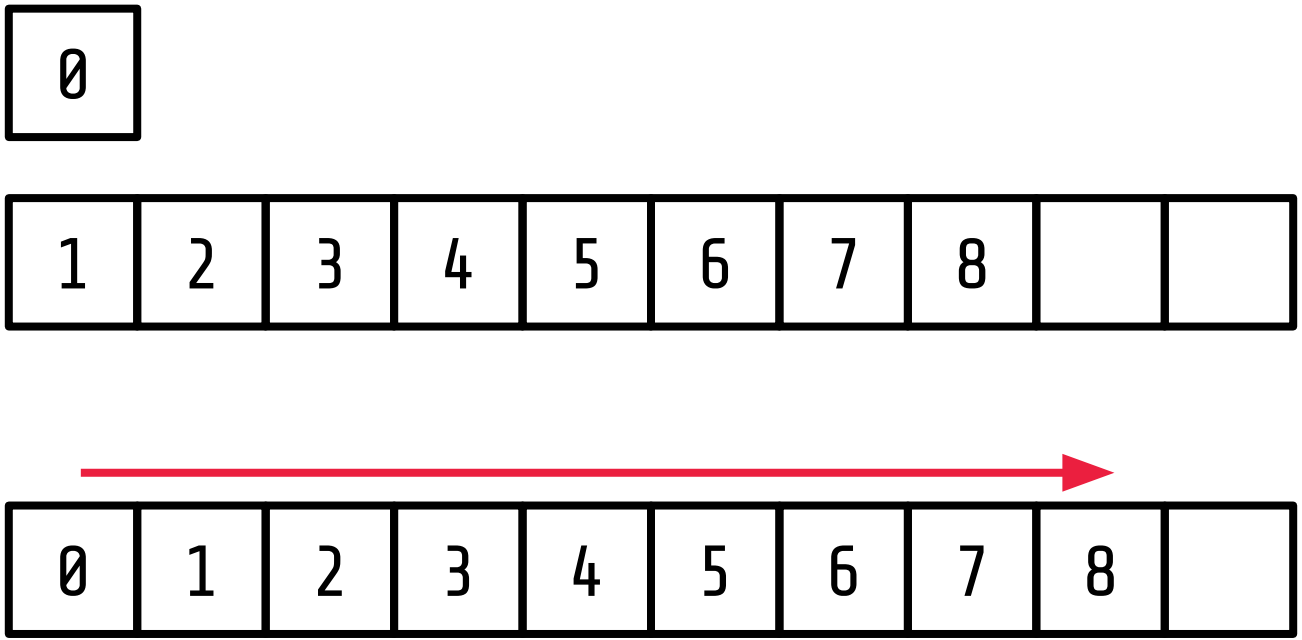




Listas vs Arreglos

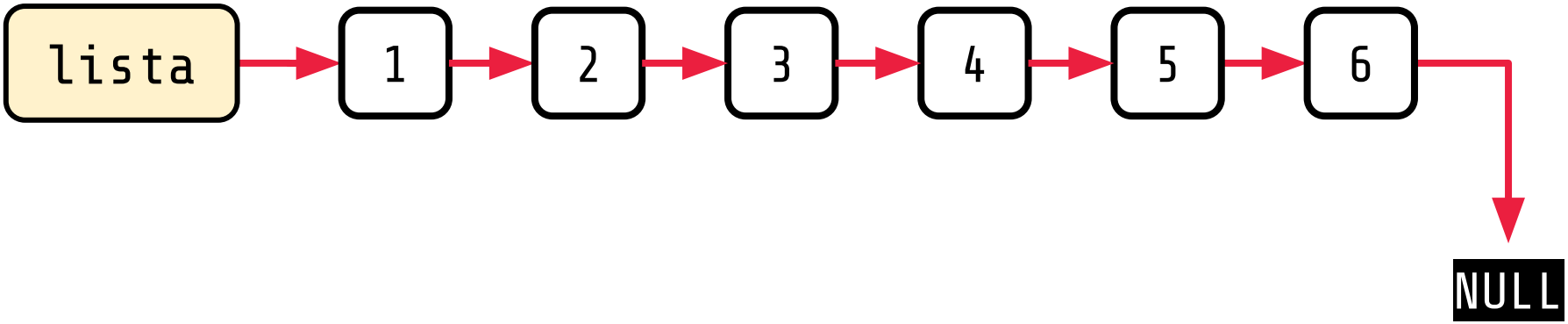


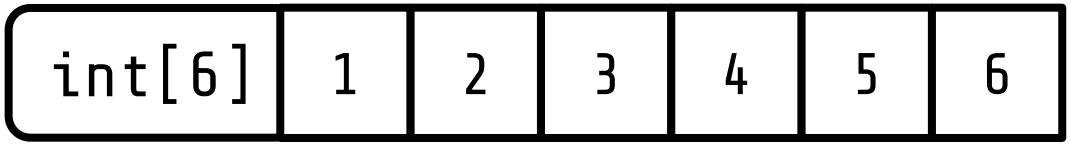
Comparado con un arreglo



Qué es necesario ‘hacer lugar’

Accesos
Modificaciones
Ampliaciones
Reducciones


lista
1
2
3
4
5
6
NULL


int[6]
1
2
3
4
5
6

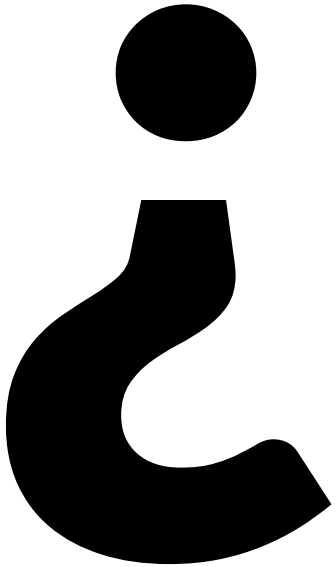
en definitiva

Las mismas operaciones que las de arreglos;

(estáticos y dinámicos)

inserción*
eliminación*
modificación
obtención


¿Preguntas?



Para que hay dos formas de secuencia



¿Para qué están?

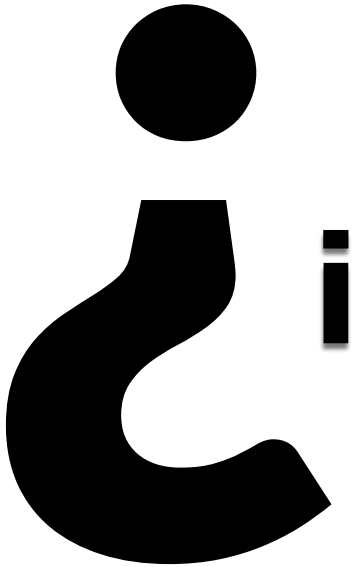
¡Complejidad computacional!

UNRN | Universidad Nacional de Río Negro

{Qué vamos a ver en un ratito}


¿Preguntas?

La mágia de la abstracción

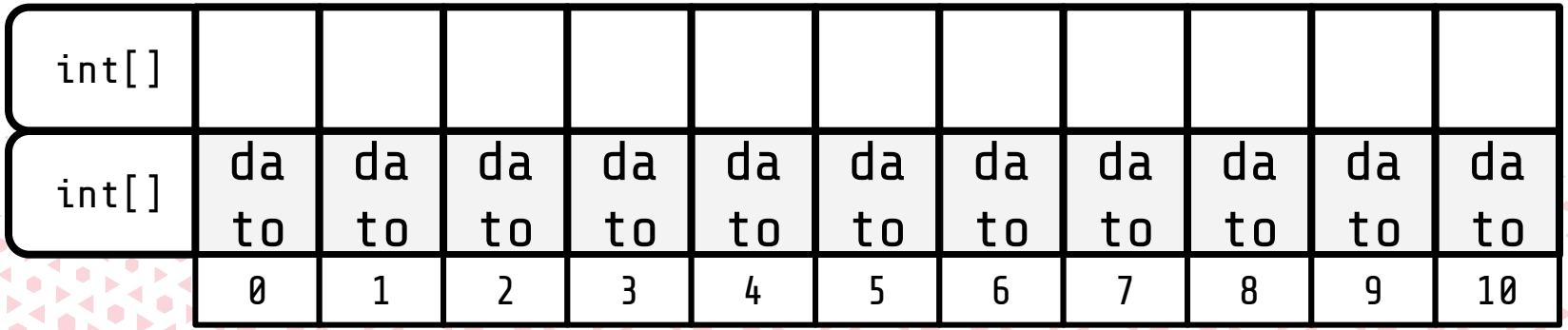


Es la única implementación



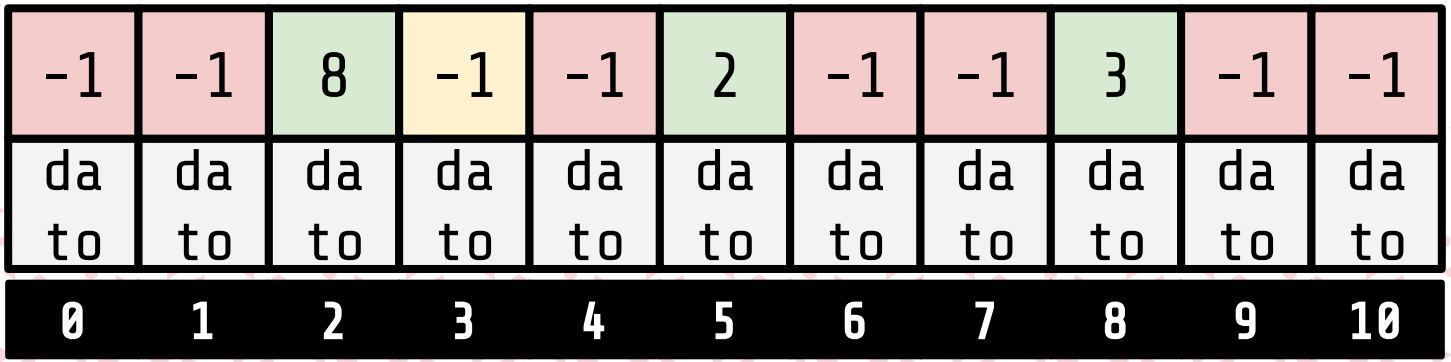
Enlaces con arreglos

inicio	int



Enlaces con arreglos

inicio	5

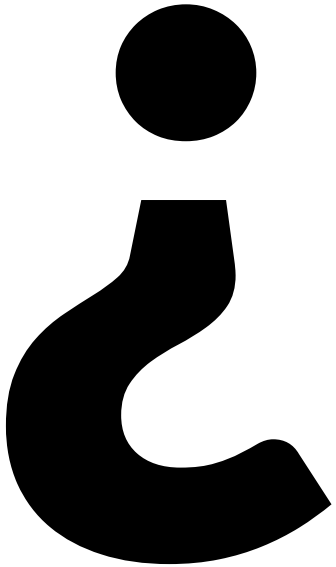


Se puede implementar con estructuras

```
typedef struct
{
    int siguiente;
    int valor;
}nodo_t;
```

```
typedef struct
{
    int nodo_t[MAX];
    int inicio;
    int capacidad;
}lista_t;
```

*

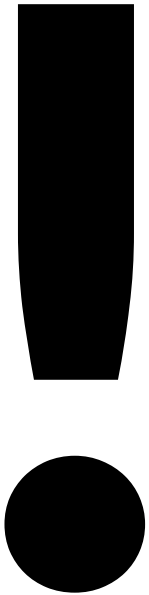


Qué ventajas y desventajas tiene cada una





La complejidad queda *contenida*




¿Preguntas?

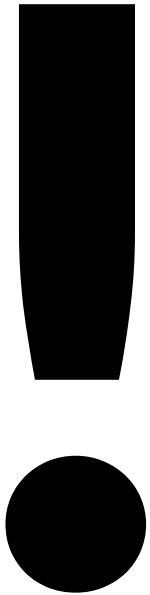
Dada esta función

```
int insertar(lista_t *lista, int posicion, int valor)
```

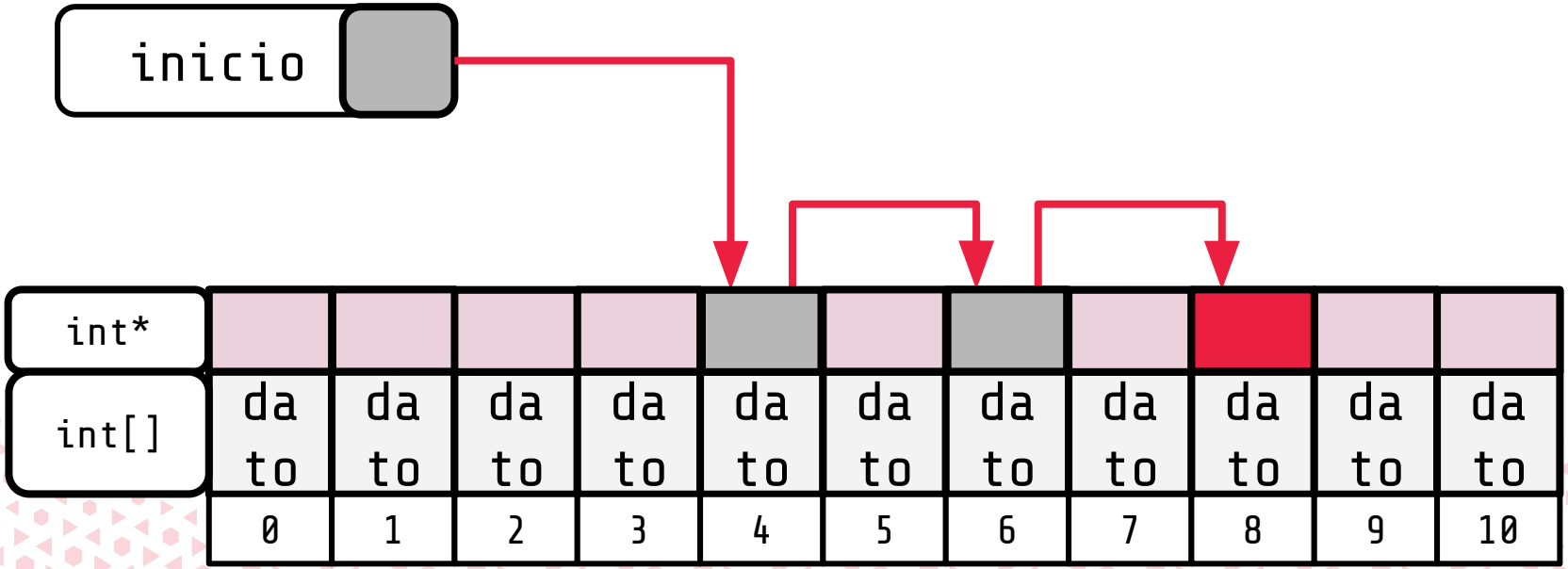
¿Podemos saber qué implementación es?



Esto es *encapsulamiento*



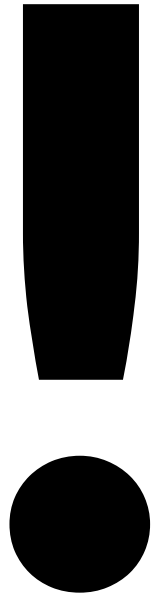
Y hay aún más formas...



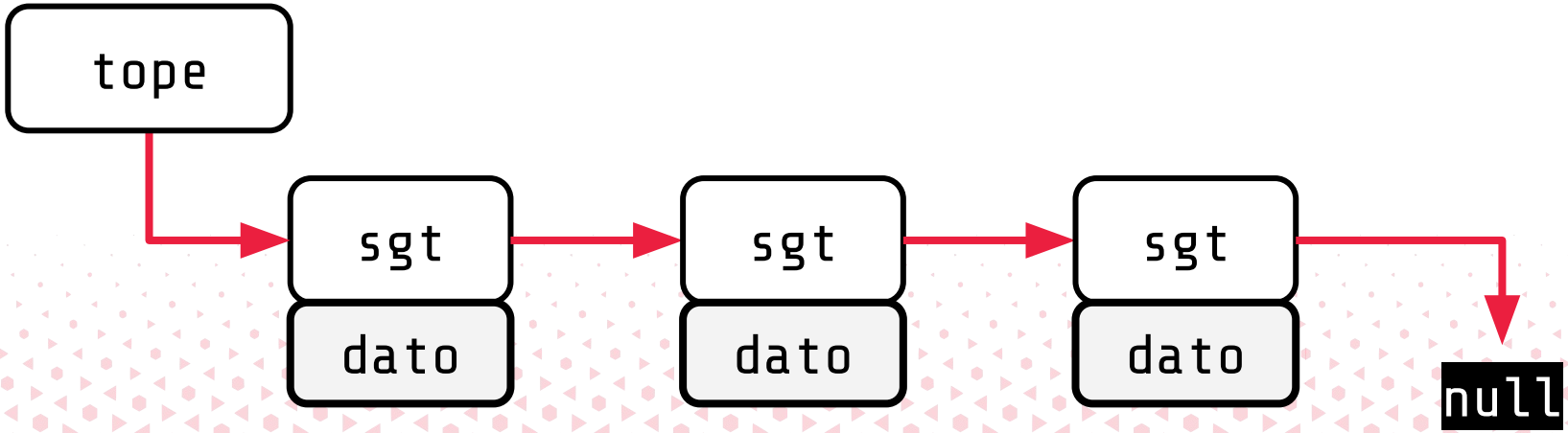

¿Preguntas?



Es la base de otras estructuras



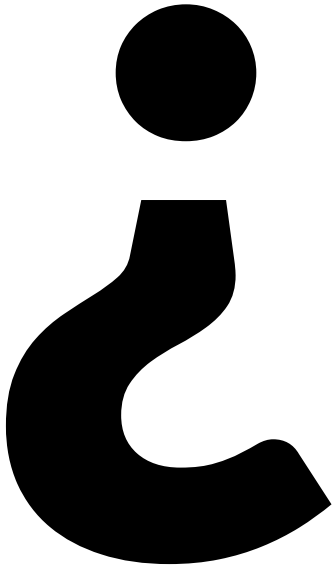
Pila (stack)



Lo último en entrar es lo primero en salir [LIFO]

Operaciones básicas en pilas

Agrega a la pila	`push(q, e)`
Toma de pila	`e = pop(q)`
Mira el frente	`e = peek(q)`
Revisa si está vacía	`is_empty(q)`
Revisa si está llena	`is_full(q)*`



Cómo se implementa



Pila basada en nodos

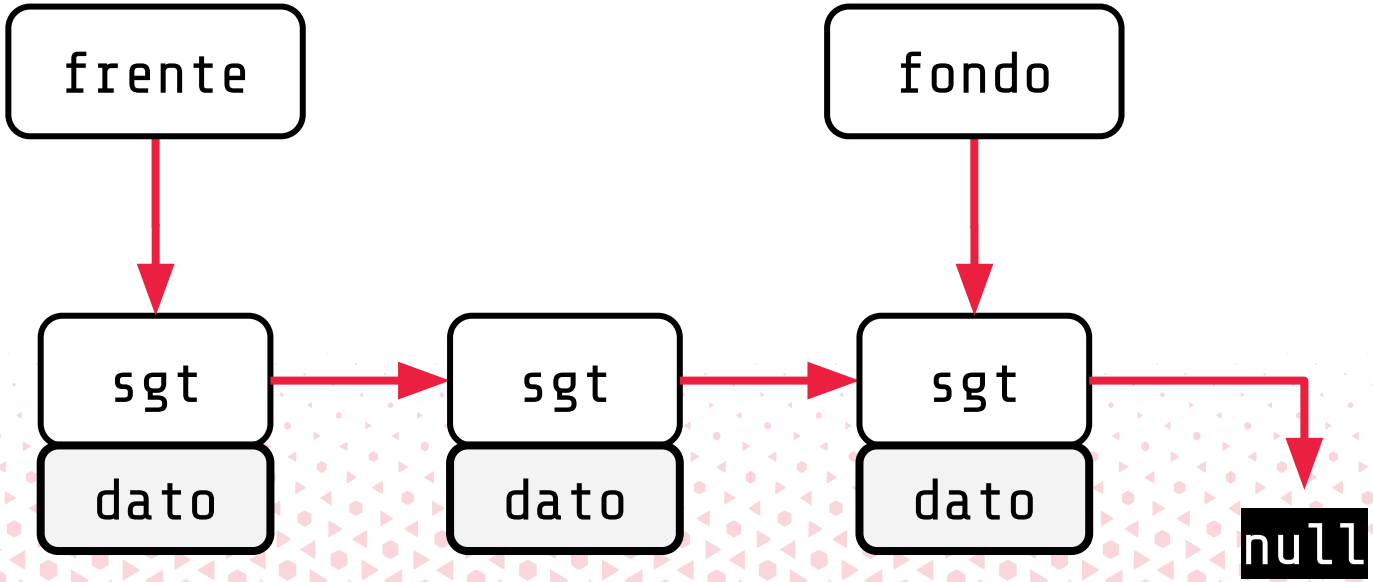
```
typedef struct {
    nodo_t *tope;        // Puntero al nodo superior
} stack_t;
```

Implementación con Arreglos

```
// Estructura de Pila basada en Arreglo
typedef struct {
    int *elementos;     // Arreglo dinámico de elementos
    int capacidad;      // Capacidad máxima de la pila
    int tope;           // Índice del elemento en el tope
                        //   (-1 si está vacía)
} stack_t;
```


¿Preguntas?

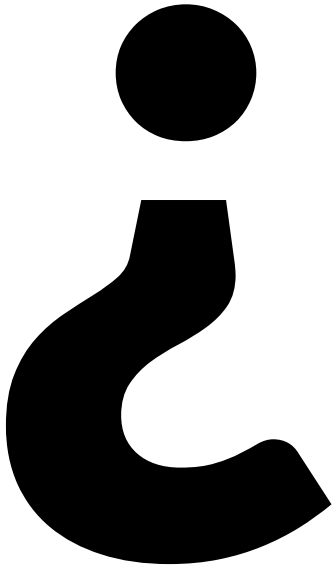
Cola (queue)



Lo primero en entrar es lo primero en salir [FIFO]

Operaciones básicas en filas

Agrega a la fila	`enqueue(q, e)`
Toma de fila	`e = dequeue(q)`
Mira el frente	`e = peek(q)`
Revisa si está vacía	`is_empty(q)`
Revisa si está llena	`is_full(q)*`



Cómo se implementa



Implementación con Arreglos*

```
// Estructura de Cola basada en Arreglo Circular
typedef struct {
    int *elementos;     // Arreglo de elementos
    int capacidad;      // Capacidad del arreglo

    int frente;         // Índice del primer elemento
    int final;          // Índice donde se vá el próximo
    int tamano;         // Cantidad actual de elementos
almacenados
} queue_t;
```

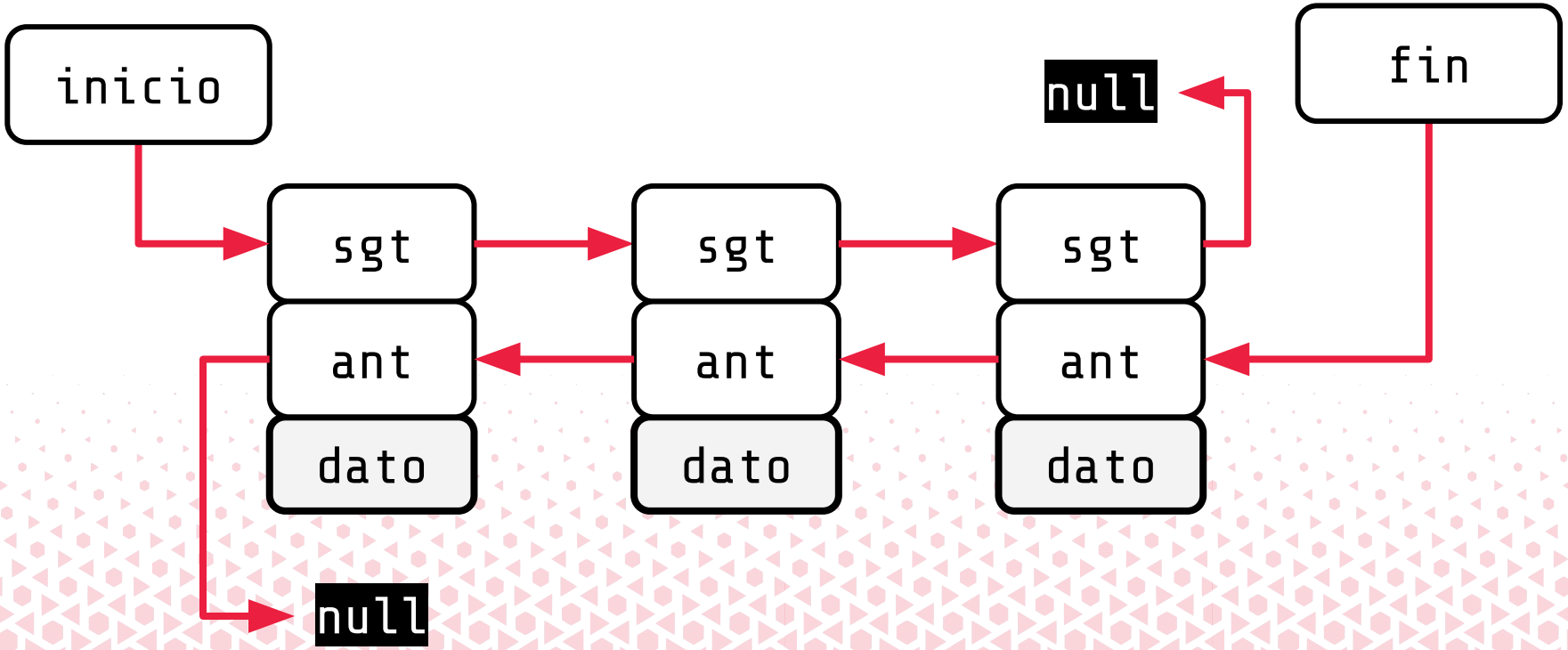
Fila basada en nodos

```
typedef struct {
    nodo_t *frente;     // Puntero al primer nodo (salida)
    nodo_t *final;      // Puntero al último nodo (entrada)
} queue_t;
```

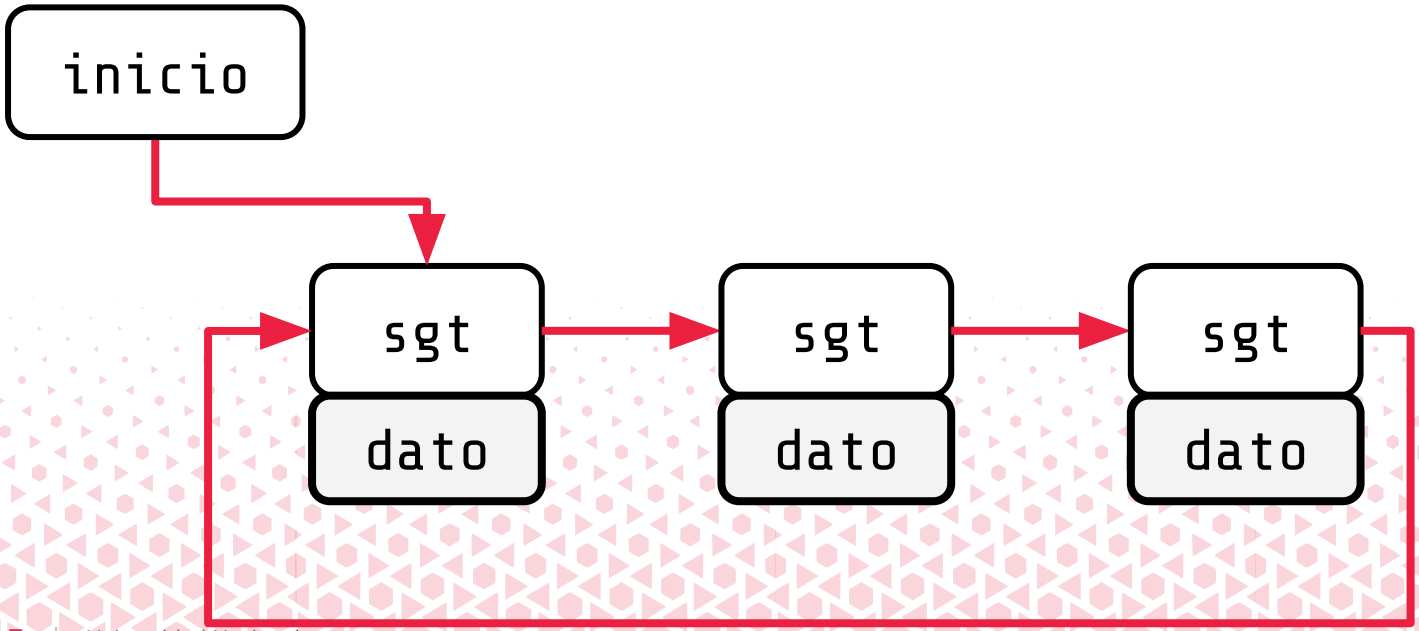

¿Preguntas?

{ Y estructuras aún más complejas }

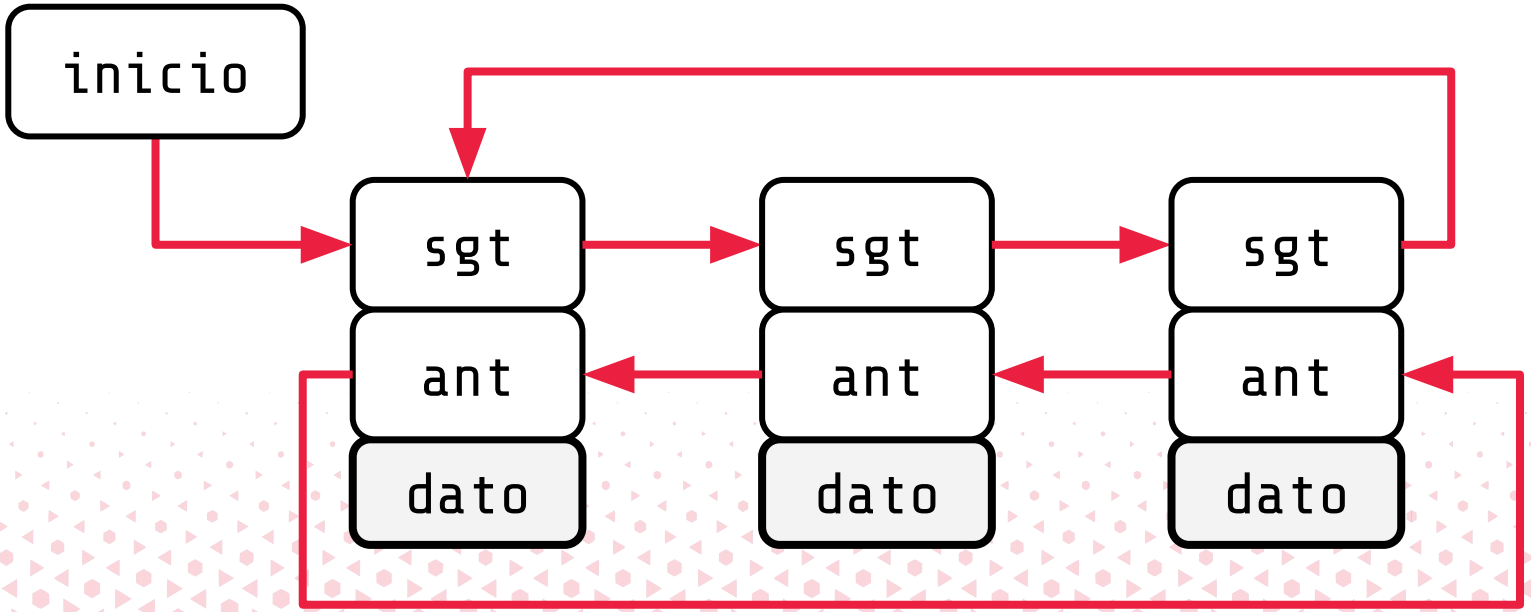
Listas doblemente enlazadas



Lista enlazada circular



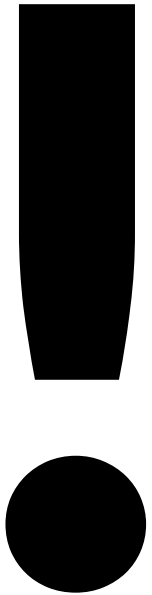
Lista enlazada doble circular




¿Preguntas?



¡Pausa!

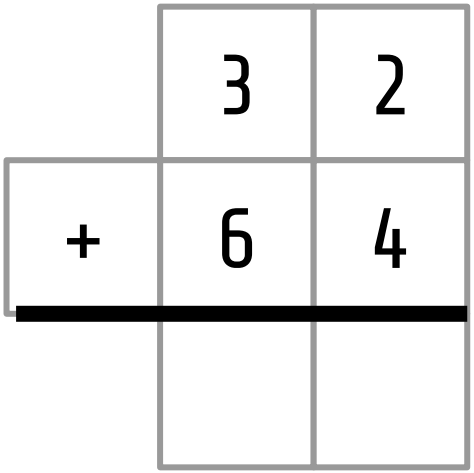


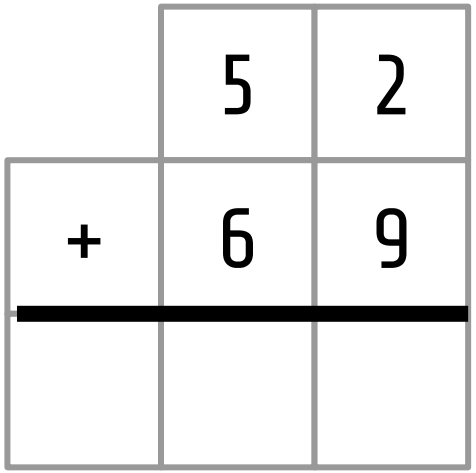
Algoritmos

UNRN Universidad Nacional de Río Negro

¿Qué son?

UNRN Universidad Nacional de Río Negro


3 2
+ 6 4


5 2
+ 6 9

Atributos



PHASE 1 PHASE 2 PHASE 3

Collect underpants

?

Profit



SOUTHPARK.CC.COM

Finito*

1 Debe terminar después de un número finito de pasos.

Definido

2

Cada paso debe ser claro y sin ambigüedad.

Eficiente*

3

Debe resolver el problema en un tiempo razonable.

Efectivo

4

Cada paso del algoritmo debe ser realizable y simple.

Partes

Entrada

E

Un algoritmo debe tener al menos una entrada, que es la información que necesita para procesar.

Salida

S

Un algoritmo debe tener al menos una salida, que es su resultado.



Y así, *nos remontamos a la clase #2*, de funciones

Formas de representación

1. Pseudocódigo
2. Diagramas de flujo
3. Código en un lenguaje específico


¿Preguntas?

Complejidad

introducción a

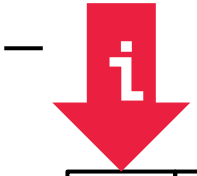
UNRN Universidad Nacional de **Río Negro**

¿Qué es?

UNRN Universidad Nacional de Río Negro

Ejemplo: **Búsqueda**

UNRN Universidad Nacional de Río Negro



0	5	10	15	20	25	30	35	40	45	50	55	60	65	70
0	1	2	3	4	5	6	7	8	9	10	11	12	13	14

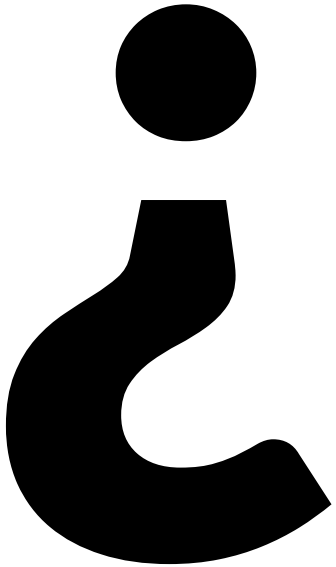
1. ¿el elemento `i` es el que busco?
2. `i = i + 1`
3. Repetir

   (hasta encontrar o agotar el conjunto)



0	5	10	15	20	25	30	35	40	45	50	55	60	65	70
0	1	2	3	4	5	6	7	8	9	10	11	12	13	14

1. i = largo / 2
2. ¿el elemento `i` es mayor a que busco?
   a. `i = (i / 2) + i`
3. ¿el elemento `i` es menor a que busco?
   a. i = i / 2
4. Repetir
   (hasta encontrar o agotar el conjunto)



Qué diferencia, no



¿Qué tipos de complejidades hay?

1

Complejidad temporal

(cuánto tiempo cuesta)

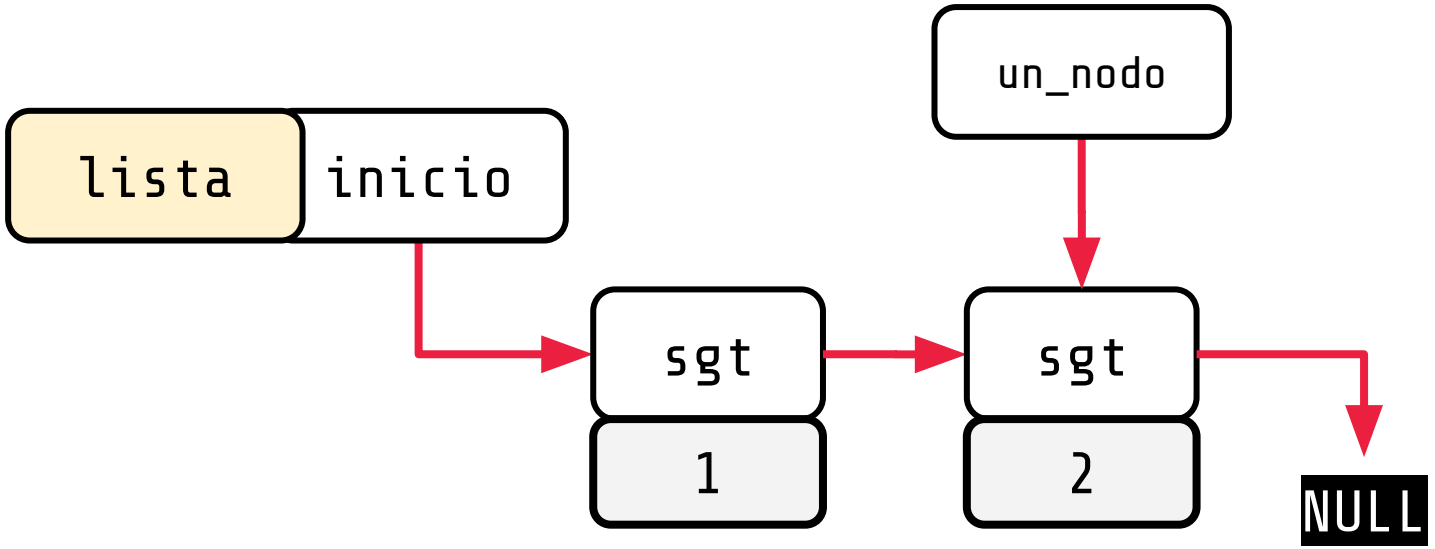
2

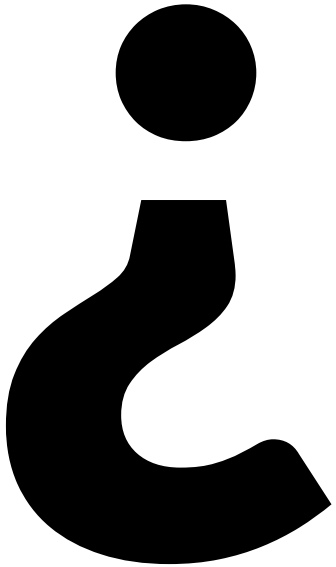
Complejidad espacial

(cuánto espacio cuesta)

{ *No se habla tanto de esta por ser* ***fácilmente*** *observable* }

¿Cuánto se ‘gasta’ extra en una lista?





Para qué se diferencian




¿Preguntas?

Notación $\mathcal{O}$

oh grande

O Cota superior
Peor caso

Describe el límite máximo de crecimiento de una función. Garantiza que el algoritmo no tardará más que ese límite.

$O(n)$

Para la búsqueda secuencial

$O(\log n)$

Para la búsqueda binaria

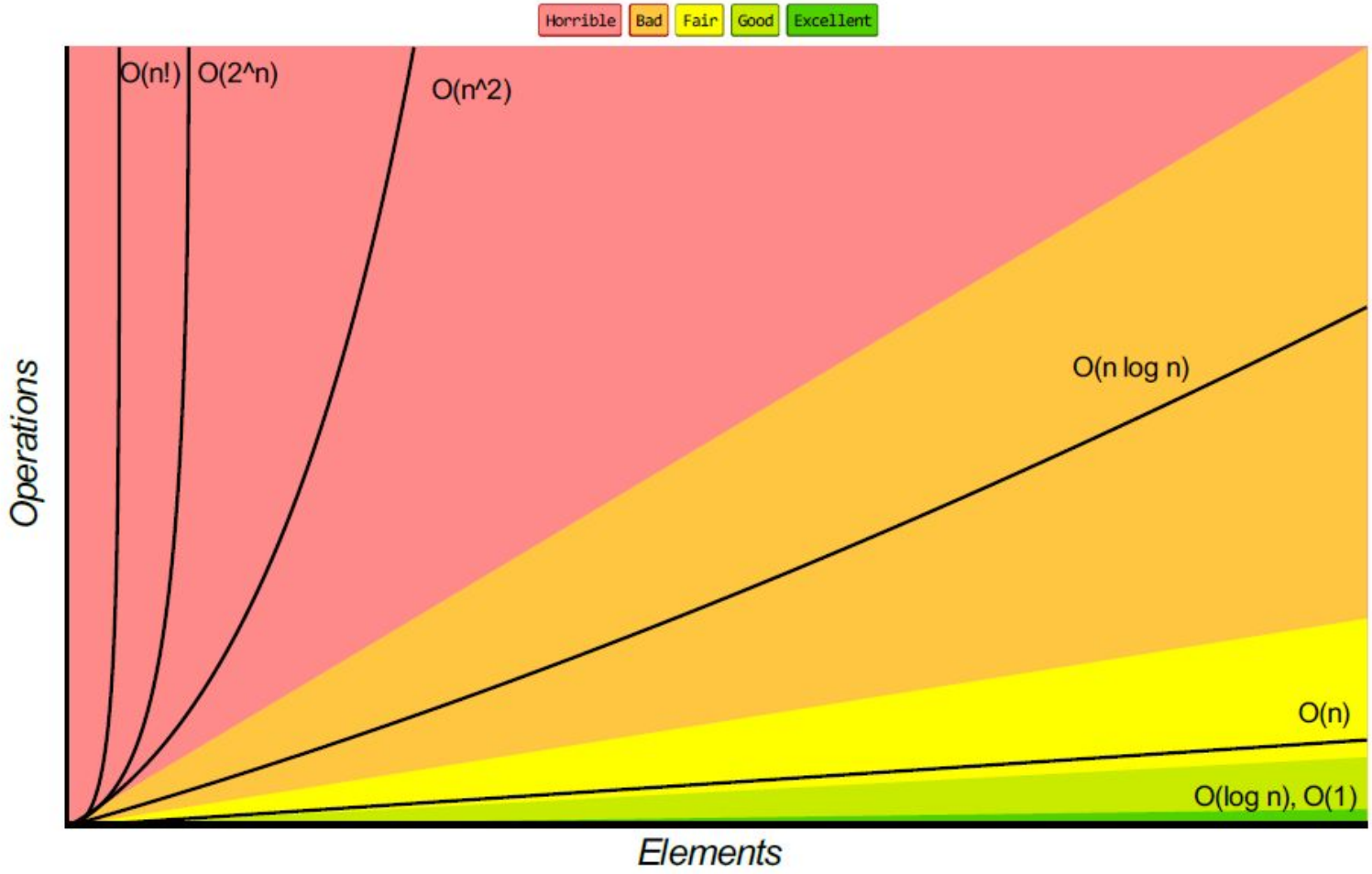
Algunas

Clases de complejidad

$O(1)$ - constante
$O(\log n)$ - logarítmica
$O(n)$ - lineal
$O(n^2)$ - cuadrática
$O(n!)$ - factorial
$O(2^n)$ - exponencial

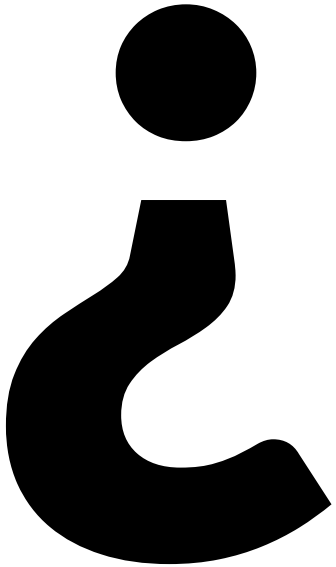
Análisis asintótico

UNRN Universidad Nacional de Río Negro



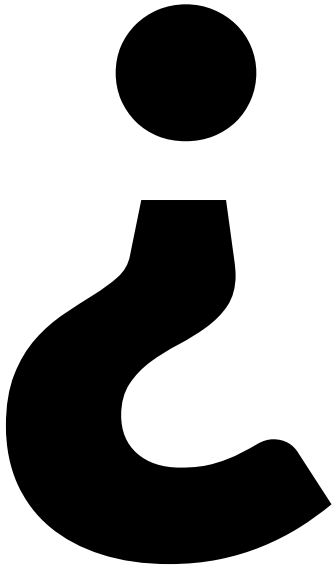
Complejidad con n grande


¿Preguntas?



Por qué vemos esto





Que significa

intuitivamente



O(1) - Constante

El tiempo no cambia aunque aumentes los datos

O(log n) - Logaritmico

Duplicar la entrada solo añade un paso más

O(n) - Proporcional

Si los datos se duplican, el tiempo se duplica

O(n log n) - Cuasilineal

Un problema dividido en subproblemas logarítmicos con trabajo lineal para terminar

O(n²) - Cuadratico

significa que cada vez que se examina un elemento hay que examinar también todos los demás.
(Si los datos se duplican, el tiempo se cuadruplica)

$O(2^n)$ - Exponencial

Es necesario probar **todas** las combinaciones de esos n elementos.

(Fibonacci recursivo*)


¿Preguntas?

¿Cómo aplicamos esto a lo que hemos implementado?

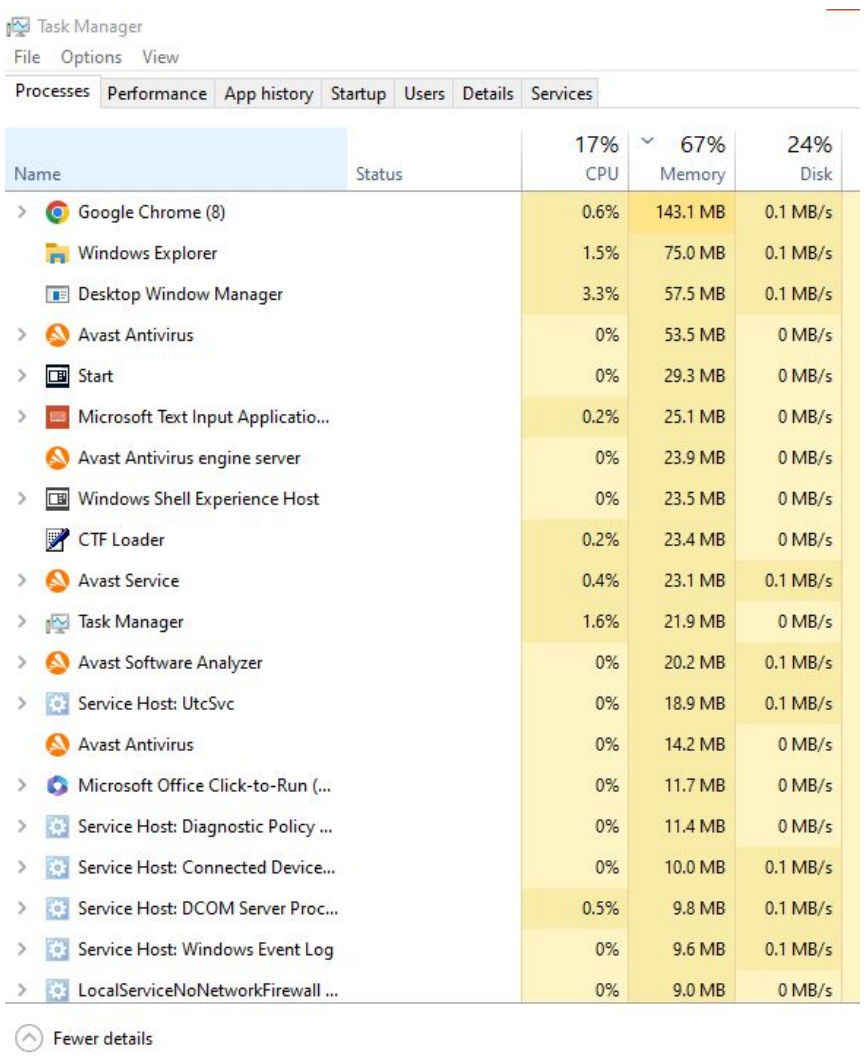
1

Podemos medir el tiempo de CPU que le lleva completar la tarea

Cronómetro
wall time



CPU Time

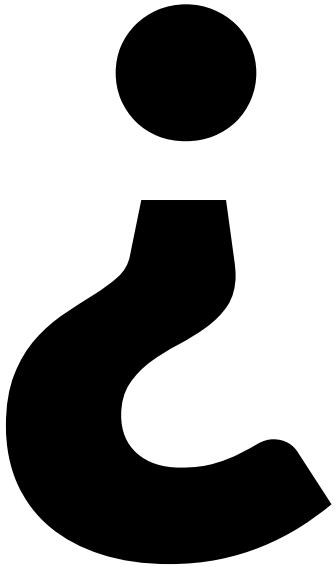

Task Manager
File Options View
Processes Performance App history Startup Users Details Services
Name Status 17% CPU 67% Memory 24% Disk
Google Chrome (8) 0.6% 143.1 MB 0.1 MB/s
Windows Explorer 1.5% 75.0 MB 0.1 MB/s
Desktop Window Manager 3.3% 57.5 MB 0.1 MB/s
Avast Antivirus 0% 53.5 MB 0 MB/s
Start 0% 29.3 MB 0 MB/s
Microsoft Text Input Applicatio... 0.2% 25.1 MB 0 MB/s
Avast Antivirus engine server 0% 23.9 MB 0 MB/s
Windows Shell Experience Host 0% 23.5 MB 0 MB/s
CTF Loader 0.2% 23.4 MB 0 MB/s
Avast Service 0.4% 23.1 MB 0.1 MB/s
Task Manager 1.6% 21.9 MB 0 MB/s
Avast Software Analyzer 0% 20.2 MB 0.1 MB/s
Service Host: UtcSvc 0% 18.9 MB 0.1 MB/s
Avast Antivirus 0% 14.2 MB 0 MB/s
Microsoft Office Click-to-Run (... 0% 11.7 MB 0 MB/s
Service Host: Diagnostic Policy ... 0% 11.4 MB 0 MB/s
Service Host: Connected Device... 0% 10.0 MB 0.1 MB/s
Service Host: DCOM Server Proc... 0.5% 9.8 MB 0.1 MB/s
Service Host: Windows Event Log 0% 9.6 MB 0.1 MB/s
LocalServiceNoNetworkFirewall ... 0% 9.0 MB 0 MB/s
Fewer details

Pero vamos a obtener toda clase de mediciones

{ Depende del Hardware y que estamos haciendo }

2

Podemos contar las operaciones elementales que requiere



Qué es una operación elemental



Contando operaciones elementales

Se suele llamar **complejidad algorítmica**.

Se trata de acotar la cantidad de operaciones elementales que toma resolver un problema en función del tamaño de la entrada.


¡Independiente de
la computadora!

Operaciones elementales

1

Todas las operaciones aritméticas sobre enteros y números de punto flotante.

Operaciones elementales

2

Las operaciones de acceso a elementos específicos en un arreglo.

Operaciones elementales

3

La lectura o escritura de una variable.

Les asignaremos costo 1

Lazos - $O(1)$

1

Si el lazo va siempre hasta un valor constante c, el costo del mismo es fijo.

Lazos - $O(1)$

```
for (int i = 0; i <= c; i++)
{
    //operaciones O(1)
}
```

Lazos - $O(n)$

2

Si el lazo va hasta un valor variable n, y contiene operaciones $O(1)$ dentro, el mismo tendrá un costo de $O(n)$

Lazos - O(n)

```
for (int i = 0; i <= n; i++)
{
    //operaciones O(1)
}
```

Lazos - O(n)

```
for (int i = 0; i <= n; i++)
{
    //operaciones O(1)
}
```

Lazos - $O(n)$

```
for (int i = 0; i <= c * n; i++)
{
    //operaciones O(1)
}
```

El valor c que multiplica a n, queda absorbido.

Lazos - $O(\log n)$

3

Si el lazo va hasta un valor variable n, contiene operaciones $O(1)$ dentro, y la iteración aumenta en un factor de 2. El mismo tendrá un costo de $O(\log n)$

Lazos - O(log n)

```
for (int i = 0; i <= n; i = i * 2)
{
    //operaciones O(1)
}
```

En incremento y decremento

Lazos - $O(n^2)$

4

Si el lazo contiene otro en donde ambos van hasta n.

Lazos - $O(n^2)$

```
for (int i = 0; i <= n; i++)
{
    for (int j = 0; j <= m; j++)
    {
        //operaciones O(1)
    }
}
```

{ Cada lazo anidado sube el valor del exponente }

Lazos - $O(n+m)$

5

La complejidad de dos lazos consecutivos, es la suma de los términos de ambos.

Lazos - $O(m+n)$

```
for (int i = 0; i < m; i++) {
    // Operaciones O(1)
}
for (int j = 0; j < n; j++) {
    // Operaciones O(1)
}
```


¿Preguntas?

Verificación de palíndromo

```
bool es_palindromo(char palabra[]) {
    int inicio = 0;                                  // O(1)
    int fin = strlen(palabra) - 1;                   // O(n)

    bool es_pal = true;                              // O(1)

    while (inicio < fin) {                           // O(n/2)
        if (palabra[inicio] != palabra[fin]) {       // O(1)
            es_pal = false;                          // O(1)
            break;                                   // O(1)
        }
        inicio++;                                    // O(1)
        fin--;                                       // O(1)
    }

    return es_pal;                                   // O(1)
}
```


¿Preguntas?

Condicionales

1

Se toma la complejidad más alta en un condicional.

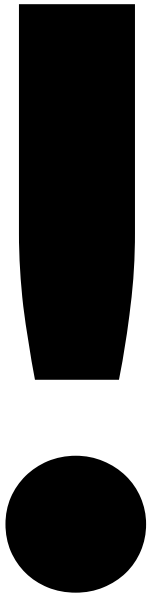
Se toma la ‘rama’ más costosa, pero una sola vez.

```
if (condicion)
{
    // instrucciones O(1)
}
else
{
    // instrucciones O(n)
}
```

{ **Pero, el análisis asintótico *simplifica* los términos** }



Quedando en las clases de complejidad *generales*




¿Preguntas?

Casos de analisis

UNRN Universidad Nacional de Río Negro

O Cota superior
Peor caso

Describe el límite máximo de crecimiento de una función. Garantiza que el algoritmo no tardará más que ese límite.

Ω Cota inferior
Mejor caso

Describe el límite mínimo de crecimiento de una función. Garantiza que el algoritmo no tardará al menos ese límite.

Cota ajustada
Comportamiento exacto

Describe el comportamiento exacto del algoritmo al acotarlo tanto por arriba como por abajo por la misma función.

Ejemplo $O(f(n)) = \Theta(f(n))$

```
int suma_elementos(int arreglo[], int n) {
    int suma = 0;                          // O(1)
    for (int i = 0; i < n; i++) {
        suma += arreglo[i];                // O(n)
    }
    return suma;                           // O(1)
}
```

Pase lo que pase, se recorren los n elementos

$O(f(n)) \neq \Theta(f(n))$

```
int busqueda_lineal(int arr[], int n, int buscado) {
    for (int i = 0; i < n; i++) {  // O(n)
        if (arr[i] == buscado) {   // O(1)
            return i;              // O(1)
        }
    }
    return -1;                     // O(1)
}
```


¿Preguntas?

fast/slow pointer


¿Preguntas?

Hasta la próxima

UNRN Universidad Nacional de **Río Negro**

unrn.edu.ar

UNRN Universidad Nacional de Río Negro

| unrionegro